Stable Internships

AlgoExpert · Medium · Famous Algorithms

The problem

- N interns and n teams each rank the other side; produce a stable matching (Gale-Shapley with interns proposing).
- Returns pairs [intern, team].

Intuition

- Gale–Shapley: interns propose in preference order; teams keep their favourite so far.

Brute force vs optimal

- Brute force: check all n! matchings for stability
- Gale–Shapley proposals → $O(n^2)$
- Precompute team rankings for O(1) comparisons

Approach

- 1. Queue of free interns
- 2. Each proposes to the next team on their list
- 3. The team keeps the better intern
- 4. Rejected interns re-queue

Worked example

intern preferences

intern 0	0	1	2
intern 1	0	2	1
intern 2	1	2	0

team preferences

team 0	2	1	0
team 1	0	1	2
team 2	0	1	2

matches:

team 0 ↔ intern 0

team 1 ↔ —

team 2 ↔ —

free: [1, 2]

Intern 0 proposes to team 0: team 0 was free → accepts intern 0

Worked example

intern preferences

intern 0	0	1	2
intern 1	0	2	1
intern 2	1	2	0

team preferences

team 0	2	1	0
team 1	0	1	2
team 2	0	1	2

matches:

team 0 ↔ intern 1

team 1 ↔ —

team 2 ↔ —

free: [2, 0]

Intern 1 proposes to team 0: team 0 prefers 1 over 0 → 0 is freed

Worked example

intern preferences

intern 0	0	1	2
intern 1	0	2	1
intern 2	1	2	0

team preferences

team 0	2	1	0
team 1	0	1	2
team 2	0	1	2

matches:

team 0 ↔ intern 1

team 1 ↔ intern 2

team 2 ↔ —

free: [0]

Intern 2 proposes to team 1: team 1 was free → accepts intern 2

Worked example

intern preferences

intern 0	0	1	2
intern 1	0	2	1
intern 2	1	2	0

team preferences

team 0	2	1	0
team 1	0	1	2
team 2	0	1	2

matches:

team 0 ↔ intern 1

team 1 ↔ intern 0

team 2 ↔ —

free: [2]

Intern 0 proposes to team 1: team 1 prefers 0 over 2 → 2 is freed

Worked example

intern preferences

intern 0	0	1	2
intern 1	0	2	1
intern 2	1	2	0

team preferences

team 0	2	1	0
team 1	0	1	2
team 2	0	1	2

matches:

team 0 ↔ intern 1

team 1 ↔ intern 0

team 2 ↔ intern 2

free: []

Intern 2 proposes to team 2: team 2 was free → accepts intern 2

Java solution

```java
public class StableInternships {
    public static int[][] stableInternships(int[][] interns, int[][] teams) {
        int n = interns.length;
        int[][] teamRank = new int[n][n];
        for (int t = 0; t < n; t++) for (int r = 0; r < n; r++) teamRank[t][teams[t][r]] = r;
        int[] teamMatch = new int[n], next = new int[n];
        java.util.Arrays.fill(teamMatch, -1);
        java.util.ArrayDeque<Integer> free = new java.util.ArrayDeque<>();
        for (int i = 0; i < n; i++) free.add(i);
        while (!free.isEmpty()) {
            int i = free.poll(), t = interns[i][next[i]++];
            int cur = teamMatch[t];
            if (cur == -1) teamMatch[t] = i;
            else if (teamRank[t][i] < teamRank[t][cur]) { teamMatch[t] = i; free.add(cur); }
            else free.add(i);
        }
        int[][] out = new int[n][];
        for (int t = 0; t < n; t++) out[teamMatch[t]] = new int[]{teamMatch[t], t};
        return out;
    }
}
```

Complexity

- Time: O(n^2)
- Space: O(n^2)

Edge cases & pitfalls

- Precompute rank[team][intern] for O(1) comparisons
- Result is intern-optimal
- Always terminates with a perfect matching

Interview tips & follow-ups

- Explain why it's stable (no blocking pair)
- Follow-up: who benefits — the proposing side

Recap

- Gale–Shapley: interns propose in preference order; teams keep their favourite so far.
- Complexity: O(n^2)